

1. C
2. A
3. A
4. B
5. A
6. A
7. B
8. A
9. B
10. B

1. **Illustrating Life Cycle of a Thread**

Write a Java program to create a thread and demonstrate its **life cycle states** such as **New, Runnable, Running, Waiting/Blocked, and Terminated** using appropriate thread methods such as `start()`, `sleep()`, and `join()`. Display suitable messages to show the state transitions.

2. **Thread Priorities**

Write a Java program to create **three threads** representing **High, Medium, and Low priority tasks**. Assign different priorities using `setPriority()` and execute the threads. Display the thread name and priority during execution to illustrate the effect of **thread priorities**.

3. **Synchronized Method**

Write a Java program to simulate a **bank account** accessed by multiple threads. Create a `deposit()` and `withdraw()` operation using a **synchronized method** so that multiple threads cannot modify the account balance simultaneously. Display the final account balance.

4. **Synchronized Block**

Write a Java program to simulate **ticket booking** where multiple threads attempt to book tickets simultaneously. Use a **synchronized block** to ensure that only one thread can access the critical section at a time. Display the booking details and remaining tickets.

5. **Static Synchronization**

Write a Java program to simulate a **shared college examination counter** accessed by multiple threads created from different objects of the same class. Use **static synchronization** to ensure that only one thread at a time can update the shared counter. Display the counter value after all threads complete.

6. **Inter-Thread Communication**

Create Customer class with `deposit()` and `withdraw()` as synchronized methods. Declare `AccountNo`, `AccName` and `Balance` as Instance Variables inside the class. From the main class, Input the amount for `withdraw()` operation and if requested amount is not available in existing `Balance` amount, `withdraw()` method should be temporarily suspended using `wait()` method until `deposit()` method receives the input for amount, to be added in the existing `Balance` amount and then `withdraw()` would be completed in a successful manner. Develop the above scenario using Synchronization and Inter-Thread Communication.

1. Illustrating Life Cycle of a Thread

```
Online Java Compiler.
Code, Compile, Run and Debug java program online.
Write your code in this editor and press "Run" button to execute it.

class Main {

    static class MyThread extends Thread {

        public void run() {
            System.out.println("Running");
            try {
                Thread.sleep(1000);
                System.out.println("Waiting");
            } catch (Exception e) {}
        }

    }

    public static void main(String[] args) throws Exception {

        MyThread t = new MyThread();

        System.out.println("New");
        t.start();
        System.out.println("Runnable");

        t.join();
        System.out.println("Terminated");
    }
}
```

New
Runnable
Running
Waiting
Terminated

...Program finished with exit code 0
Press ENTER to exit console.

2. Thread Priorities

```
Online Java Compiler.
Code, Compile, Run and Debug java program online.
Write your code in this editor and press "Run" button to execute it.

class Main {

    static class Task extends Thread {

        public void run() {
            System.out.println(getName() + " : " + getPriority());
        }

    }

    public static void main(String[] args) {

        Task low = new Task();
        Task medium = new Task();
        Task high = new Task();

        low.setName("Low");
        medium.setName("Medium");
        high.setName("High");

        low.setPriority(3);
        medium.setPriority(5);
        high.setPriority(8);

        low.start();
        medium.start();
        high.start();
    }
}
```

Medium : 5
High : 8
Low : 3

...Program finished with exit code 0
Press ENTER to exit console.

2.

3. Synchronized Method

```
Online Java Compiler.
Code, Compile, Run and Debug java program online.
Write your code in this editor and press "Run" button to execute it.

class Main {
    static class Ticket {
        int tickets = 3;

        void book(String name) {
            synchronized (this) {
                if (tickets > 0) {
                    System.out.println(name + " booked ticket");
                    tickets--;
                    System.out.println("Tickets left: " + tickets);
                } else {
                    System.out.println(name + " - No tickets");
                }
            }
        }
    }

    public static void main(String[] args) {
        Ticket t = new Ticket();

        Thread t1 = new Thread(() -> t.book("Student 1"));
        Thread t2 = new Thread(() -> t.book("Student 2"));
        Thread t3 = new Thread(() -> t.book("Student 3"));
        Thread t4 = new Thread(() -> t.book("Student 4"));

        t1.start();
        t2.start();
        t3.start();
        t4.start();
    }
}
```

Student 1 booked ticket
Tickets left: 2
Student 2 booked ticket
Tickets left: 1
Student 3 booked ticket
Tickets left: 0
Student 4 - No tickets

...Program finished with exit code 0
Press ENTER to exit console.

4. Synchronized Block

```
Write your code in this editor and press "Run" button to execute it.

class Main {
    static class Bank {
        int balance = 1000;

        synchronized void deposit(int amount) {
            balance = balance + amount;
            System.out.println("Deposited: " + amount);
        }

        synchronized void withdraw(int amount) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        }
    }

    public static void main(String[] args) {
        Bank b = new Bank();

        Thread t1 = new Thread(() -> b.deposit(500));
        Thread t2 = new Thread(() -> b.withdraw(300));

        t1.start();
        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (Exception e) {
            System.out.println(e);
        }

        System.out.println("Final Balance: " + b.balance);
    }
}
```

Withdrawn: 300
Deposited: 500
Final Balance: 1200

...Program finished with exit code 0
Press ENTER to exit console.

5. Static Synchronization

```
Write your code in this editor and press "Run" button to execute it.

*****/
class Main {

    static class Exam {

        static int counter = 0;

        static synchronized void update(String name) {
            counter++;
            System.out.println(name + " : Counter = " + counter);
        }

    }

    public static void main(String[] args) {

        Exam e1 = new Exam();
        Exam e2 = new Exam();

        Thread t1 = new Thread(() -> e1.update("Student 1"));
        Thread t2 = new Thread(() -> e2.update("Student 2"));
        Thread t3 = new Thread(() -> e1.update("Student 3"));

        t1.start();
        t2.start();
        t3.start();

        try {
            t1.join();
            t2.join();
            t3.join();
        } catch (Exception e) {
            System.out.println(e);
        }

        System.out.println("Final Counter = " + Exam.counter);
    }

}
```

```
Student 1 : Counter = 1
Student 2 : Counter = 2
Student 3 : Counter = 3
Final Counter = 3

...Program finished with exit code 0
Press ENTER to exit console.
```

6. Inter-Thread Communication

```
8 class Main {
9
10     static class Customer {
11         int balance = 1000;
12
13         synchronized void withdraw(int amount) {
14             while (balance < amount) {
15                 System.out.println("Insufficient Balance");
16                 try {
17                     wait();
18                 } catch (Exception e) {}
19             }
20
21             balance -= amount;
22             System.out.println("Withdrawn: " + amount);
23             System.out.println("Balance: " + balance);
24         }
25
26         synchronized void deposit(int amount) {
27             balance += amount;
28             System.out.println("Deposited: " + amount);
29             System.out.println("Balance: " + balance);
30             notify();
31         }
32     }
33
34     public static void main(String[] args) {
35
36         Customer c = new Customer();
37
38         Thread t1 = new Thread(() -> c.withdraw(1500));
39
40         Thread t2 = new Thread(() -> {
41             try {
42                 Thread.sleep(1000);
43             } catch (Exception e) {}
44             c.deposit(1000);
45         });
46
47     }
```

```
Insufficient Balance
Deposited: 1000
Balance: 2000
Withdrawn: 1500
Balance: 500

...Program finished with exit code 0
Press ENTER to exit console.
```